quality-code

From a Jira ticket to tests you can trust.
How to use it, step by step.

What it is

quality-code is a Claude Code plugin for any Jira project and any repo with Vitest or Playwright. It checks that a ticket is clear enough to test, helps its author fill the gaps, writes the tests from the ticket and grades them. **Claude writes; scripts decide.** A score, a verdict, a pass or a refusal always comes from a script, never from the model's opinion.

You want to	Command
Set up a repo (once)	`/quality-code:setup`
Start your day on a feature branch	`/quality-code:spec`
Check whether a ticket is ready to test	`/quality-code:ticket ABC-123 --eval`
Run and grade the tests on your branch	`/quality-code:check --branch`
Have tests written from a ticket	`/quality-code:spec ABC-123`
Write only the test data a ticket needs	`/quality-code:seed ABC-123`
See how the team uses it	`/quality-code:report --kpi`
See what the AI runs cost	`/quality-code:report --cost`
Turn a Teams planning meeting into tasks	`/quality-code:transcript-goal`
Run the few critical tests	`/quality-code:check --smoke`
Change one setting	`/quality-code:config set git.base develop`
Clear the cache, or undo the setup	`/quality-code:clean --cache`, `/quality-code:clean`

1. Install (each person, once)

You need: Claude Code (desktop app, terminal or IDE), Node 18 or newer, a repo whose `package.json` has `vitest` or `@playwright/test`, and access to your Jira site.

1. In a Claude Code session, inside your repo, add the marketplace and install the plugin:

```
/plugin marketplace add vissz28/agentic-testing
/plugin install quality-code@teamsystem
```

2. Open `/plugin` and check that quality-code shows version **0.66.0**. If it shows an older one, go to **Marketplaces**, pick quality-code, choose **Update marketplace** and **Enable auto-update**, then update the plugin and start a new session.
3. Connect Jira: run `/mcp`, choose `atlassian-rovo` and sign in (an Atlassian connection you already have in claude.ai works too). GitLab and Figma are optional.

2. Set up the repo (one person, once)

```
/quality-code:setup
```

1. Answer the numbered cards: the stack (Vitest or Playwright), the Jira site and project, the acceptance-criteria field. Type the number of your choice each time.
2. Setup writes `.claude/quality-code.json` only after you confirm, then proves the setup on one recent ticket.
3. Commit the config and the `.gitignore` lines it adds.
4. Later, `/quality-code:setup advanced` adds forbidden paths (code a test run must never change), feature labels, title prefixes, templates and, for Vitest, test folders.

To offer the plugin to everyone who opens the repo, commit this in `.claude/settings.json`:

```
{
  "extraKnownMarketplaces": {
    "teamsystem": { "source": { "source": "github", "repo": "vissz28/agentic-testing" } }
  },
  "enabledPlugins": { "quality-code@teamsystem": true }
}
```

3. Every day: start from your branch

```
/quality-code:spec
```

1. It checks your connectors (Jira must be connected).
2. It shows the repo card: your branch, its base branch and what the branch changed. Confirm with 1.
3. It proposes the ticket from the branch name (for example `ABC-123-add-export`). Pick **1. Use ABC-123** or **2. A different ticket**.
4. It offers what to do: **1. Evaluate the ticket**, **2. Check the tests on this branch**, **3. Write and check tests**.

4. Check a ticket: the readiness gate

```
/quality-code:ticket ABC-123 --eval
```

The ticket gets a score out of 100. **70 or more passes**, for every team. The score rewards acceptance criteria (Gherkin, or at least 150 characters of plain language under an Acceptance criteria heading), testable results (no "works correctly"), steps or a user story, the expected behaviour and the context. Customer ids, e-mails and UUIDs are redacted before scoring.

Below 70, the refuse card

Option	What happens
1. Post a comment on Jira	A comment listing exactly what to add, with examples. You see a preview first.
2. I will add the missing information here	A guided fill-in: pick the missing items, answer one short question each; the script says whether each answer counts. Then you can push the added parts to Jira (append only, preview first).
3. Continue anyway (developers only)	Nothing is posted. Inside `write`, tests are still written, but every result card opens with **[RISK / NOT RECOMMENDED] Ticket incomplete (score/70)** and the override is recorded.

In CI: `ticket ABC-123 --eval --ci` asks nothing; a refused ticket gets the Jira comment and the job fails.

5. Check the tests on your branch

```
/quality-code:check --branch                        # the test files your branch changed
/quality-code:check test/orders.spec.ts   # or name the specs
/quality-code:check --key ABC-123 --flaky 3
```

The pipeline always runs the same steps, in this order, and a script decides the verdict:

Step	It checks
boundary	during a writing run: no change outside the test folders
honesty	no assertion or test removed, no skip without a ticket
assertions	every test that runs has an expect(...)
types	the specs compile (tsc --noEmit)
tests	the tests run, with your own Vitest or Playwright
expected	with a ticket: the ticket's numbers and quoted texts are asserted
mutation	each assertion can fail when its value changes
flaky	with --flaky N: no test changes result between runs
stryker	Vitest, with --target and Stryker installed: the tests catch changes in the code

Verdict	Meaning	Exit
green	every step ran and passed	0
checks failed	a check failed: the card says what to do	1
tests red	the checks pass but tests fail (expected for a bug whose fix is not merged)	1
not verified	something could not run; never counted as green	3

Playwright: the app must be running, or started by your Playwright config's webServer. If it is down, the verdict is "not verified", never red.

6. Have tests written from a ticket

```
/quality-code:spec ABC-123
/quality-code:spec ABC-123 --target src/pricing.ts   # Vitest
/quality-code:spec ABC-123 --route /orders           # Playwright
```

1. **Gate.** The ticket is scored first. Only a ticket that passes (or option 3, marked at risk) gets tests.
2. **Target.** Vitest: which code the ticket changes. Playwright: which screen (from --route or the feature's route). Pick from the card.
3. **Plan.** What will be written (runner, libraries, file paths), then the scenarios from the ticket, one line each, and where each goes. Pick **1. Yes, write these tests**, **2. No, stop** or **3. Something else**.
4. **Write.** The specs are written only in your test folders; application code is never touched (a hook blocks it). Real expected values from the ticket, one test per scenario.
5. **Check.** The pipeline grades them. Fixable problems get at most 2 rewrites of the tests only.
6. **Card.** You get the last card as printed, and the spec paths. Read the tests before you commit them.

For a **Bug**, a red test that asserts the ticket's expected value is the right result: it goes green when the fix is merged. On Playwright, up to 3 extra probes (titled "Extra: ...") try cases the ticket did not state; `--no-extras` leaves them out.

7. Write only the test data

```
/quality-code:seed ABC-123
/quality-code:seed ABC-123 --path test/unit/orders.abc-123.spec.ts
```

Vitest: calls to your own seed builders, with the ticket's exact values. Playwright: route stubs copied from a stub you already have. It never invents a value or a response shape and never writes to a database. It prints the block, or puts it into an existing spec with --path.

8. See how it is used, and what it costs

```
/quality-code:report --kpi --since 30d
/quality-code:report --cost --by ticket
```

kpi: tickets gated, fill-ins, Jira comments, forced runs, verdicts, boundary blocks (from a local, git-ignored audit log; keys and numbers only). **consumption**: AI tokens and an estimated cost per command, ticket, day or model, read from Claude Code's local logs. Both send nothing anywhere.

More commands

Rewrite a weak story

After `ticket --eval` scores a ticket, a script checks whether the description is too long, has no structure or is too thin. If so you pick a source (the description, Figma, Confluence or your own text), see the draft as a card with its new score, and choose: post to Jira (after a preview and your yes), save locally, edit more, or cancel.

Sprint goal from a Teams transcript

```
/quality-code:transcript-goal --file planning.vtt
```

Reads a transcript file, pasted text or the meeting through the Microsoft 365 connector. It drafts the sprint goal, decisions, tasks and risks, asks only what was not said, and shows review cards. Tasks are created in Jira only after a preview and your yes; existing tickets are linked, not duplicated. Or save it locally.

More kinds of tests

What	How it works
Negative cases	spec proposes tests for invalid input, errors and denied permissions on every ticket (Negative: N cases on the plan card).
Accessibility	Each Playwright screen test gets an axe scan (WCAG 2.1 AA). It asks before installing; --no-axe turns it off.
Visual regression	Optional screenshot checks against a baseline that you confirm before it is created or updated.
Database checks	Backend tickets that touch data get constraint, migration and integrity tests on a real database (Testcontainers).
Regression from a bug	A Bug ticket becomes a @regression test that fails first and passes after the fix. routine --regression runs them.
Smoke suite	check --smoke runs the tests tagged @smoke against the app or any URL. ci --smoke adds a post-deploy job (SMOKE_URL).

Report pages with charts

```
/quality-code:report --kpi --report
/quality-code:ticket ABC --eval --report
/quality-code:report --cost --report
```

Each writes one self-contained HTML page (light and dark, no network) with charts: verdicts over time, pass rate per ticket, score bands, mutation, cost by command and day. Only keys, scores and numbers go in. The chat still shows the text card, now with a headline row (verdict, score, bar with the gate), a Next line and collapsible detail.

Settings, cache and clean-up

```
/quality-code:config set git.base develop
/quality-code:config get jira.site
/quality-code:clean --cache --all
/quality-code:clean --dry-run
```

config set changes one setting, checked by the setup rules; it shows old and new and writes nothing on an error or a secret. Tokens and the Jira email stay in /plugin. **clean --cache** deletes only the plugin's local cache. **clean** removes what the plugin created in the project (setup files, saved output, cache; your tests and the personal config only if you pick them). It always asks one explicit yes and is never skipped by --yes or --ci.

9. Bringing a team on board

Checklist for a team lead

	Done when
Plugin	everyone has quality-code 0.66.0 or later and Jira connected (start shows Jira connected)
Config	.claude/quality-code.json reviewed, committed, and someone owns it
Boundaries	the forbidden paths cover your application code (default: src/, .env, .env.*, the CI file)
Vitest	the test folders are right; Stryker installed if you want its bar
CI	a job runs ticket KEY --eval --ci, if you gate tickets in CI
Feedback	everyone knows to report a surprise as an issue with the ticket key and the card

Moving from your own harness

1. Set up quality-code and compare first: ticket --eval on recent tickets with both, check on the same specs.
2. In one merge request: add the config, remove the old harness (commands, skills, hooks, scripts, their lines in CLAUDE.md). Never keep two harnesses in one repo.
3. Replace a CI ticket gate with ticket KEY --eval --ci.

The full runbook for the warehouse repos is `docs/rollout/onboarding-quality.md`.

10. If something goes wrong

You see	Do this
An old version in /plugin	Marketplaces, quality-code, Update marketplace; then update the plugin and start a new session.
This repo is not set up	Run /quality-code:setup.
Jira not connected	Run /mcp, choose atlassian-rovo, sign in.
Another test-writing run is active	Pick 1 to stop it and start yours. A run older than 12 hours is replaced on its own.
The app is not reachable (Playwright)	Start the app, or let the webServer in your Playwright config start it, and check again.
A write is blocked by the boundary	The test tried to change application code. Tests never do; ask a person if it really belongs there.

More: the README and docs/GETTING_STARTED.md in github.com/vissz28/agentic-testing; what changed in each version is in CHANGELOG.md.